Quiz on Wed 8:30-9:10 am

- Syllabus:
  - Everything covered till Aug 18/19.
- Question paper format:
  - Fill in the blanks in a given program
  - What is the output of a given program
  - What should be the input for a desired output
  - Show last year’s question paper

Mark averaging

"Read marks of students from the keyboard and print the average."

- Number of students not given explicitly.
- If a negative number is entered as mark, then it is a signal that all marks have been entered.
- Assume at least one positive number is given.

Examples:

- Input: 98 96 -1, Output: 97
- Input: 90 80 70 60 -1, Output: 75

- Cannot be done using what you know so far.
  - `repeat` repeats fixed number of times. Not useful

  Need new statement e.g `while`, `do while`, or `for`
- `repeat`, `while`, `do while`, `for` : "looping statements".

Outline

- The while statement
  - Some simple examples
  - Mark averaging
- The break statement
- The continue statement
- The do while statement
- The for statement
- Loop invariants
  - GCD using Euclid's algorithm

The while statement

Form `while (condition) body`

1. Evaluate `condition`.
2. If `false`, execution of statement ends.
3. If `true`, execute `body`. `body` can be a single statement or a block, in which case all the statements in the block will be executed.
4. Go back and execute from step 1.

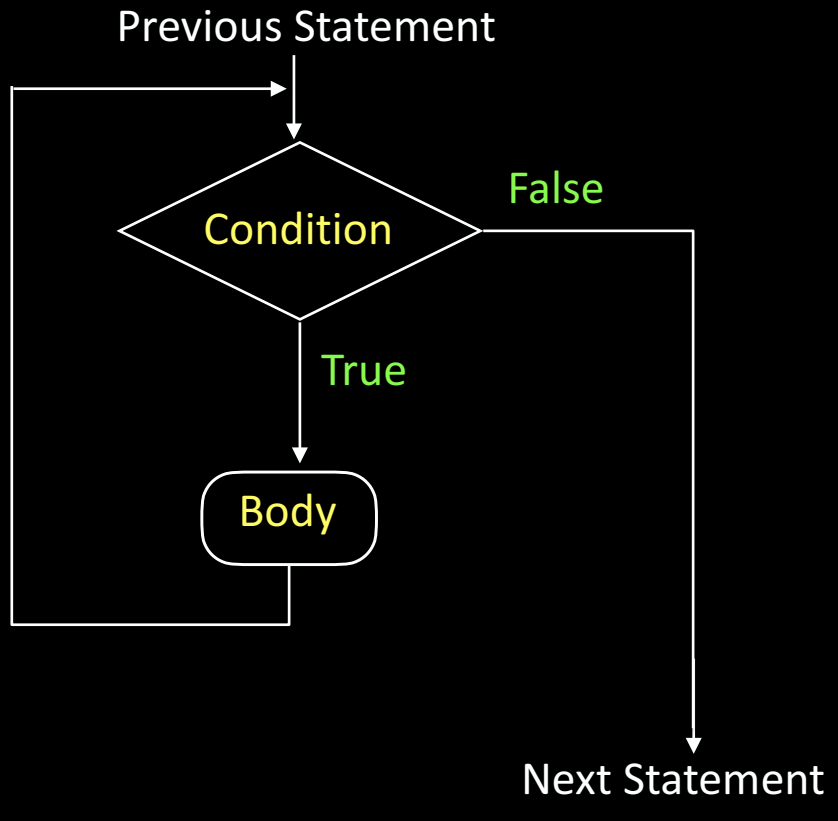
- The `condition` must eventually become `false`, otherwise the program will never halt. Not halting is not acceptable.
- If `condition` is `true` originally, then value of some variable in `condition` must change in the execution of `body`, so that eventually `condition` becomes `false`.
- Each execution of the body = iteration.

while flowchart

...previous statement...

```
while(condition) body
```

...Next statements...



A silly example

```
main_program{
   int x=2;
   while(x > 0){
      x--;
      cout << x << endl;
   }
  cout <<“Done.”<<endl;
}
```

- First x=2 is executed.
- Next, x > 0 is checked
- x=2 is > 0, so body entered.
- x is decremented, becomes 1.
- x is printed. (1)
- Back to top of loop.
- x=1 is > 0, body entered.
- x is decremented, becomes 0.
- x is printed. (0)
- Back to top of loop.
- x=0 is not > 0. body not entered.
- “Done.” printed

while vs. repeat

- Anything you can do using repeat can be done using while.

```
repeat(n){ xxx }
```

- Equivalent to

```
int i=n;
while(i>0){i--; xxx }
```

Assumption: the name i is not used elsewhere in the program.

- If it is, pick a different name

Exercise

- What will the following program fragment print?

```
int x=306, y=77, z=0;
while(x > 0){
  z = z + y;
  x--;
}
cout << z << endl;
```

- Write the following using `while`.

```
repeat(4){
  forward(100);
  right(90);
}
```

What we discussed

When to use `while` statement:

- We need to iterate while a condition holds
- We need not know at the beginning how many iterations are there.

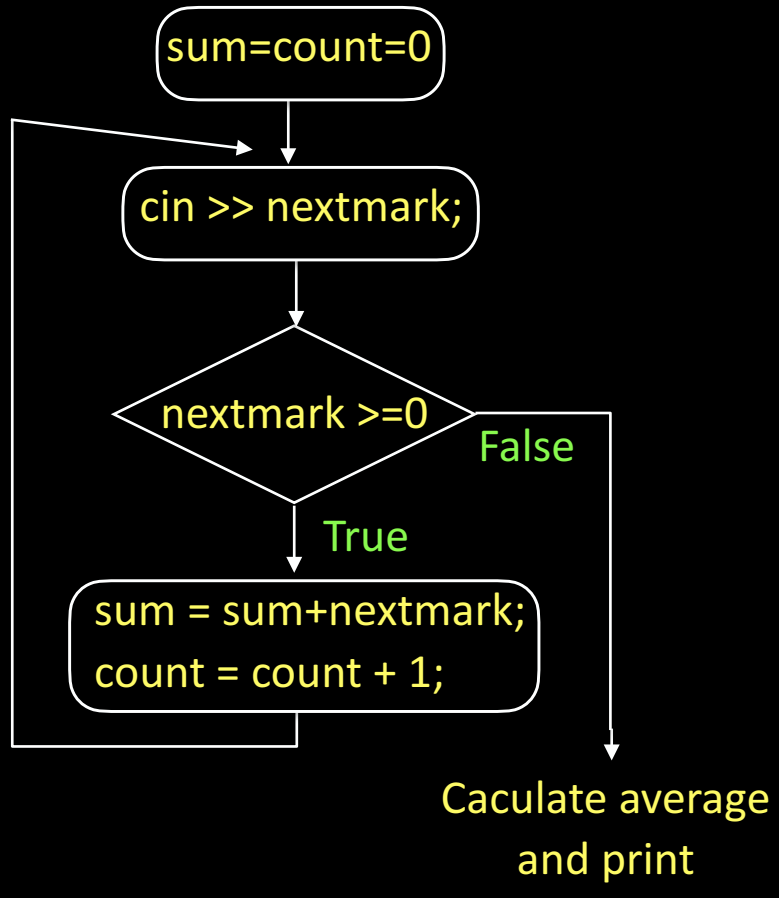
Whatever we can do using `repeat` can be done using `while`

- Converse not true

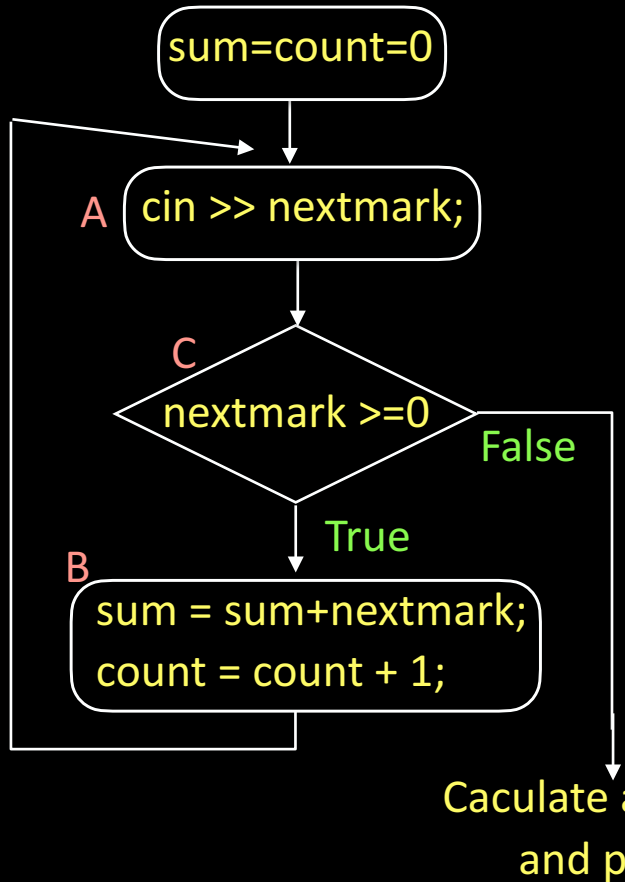
Mark averaging: manual algorithm

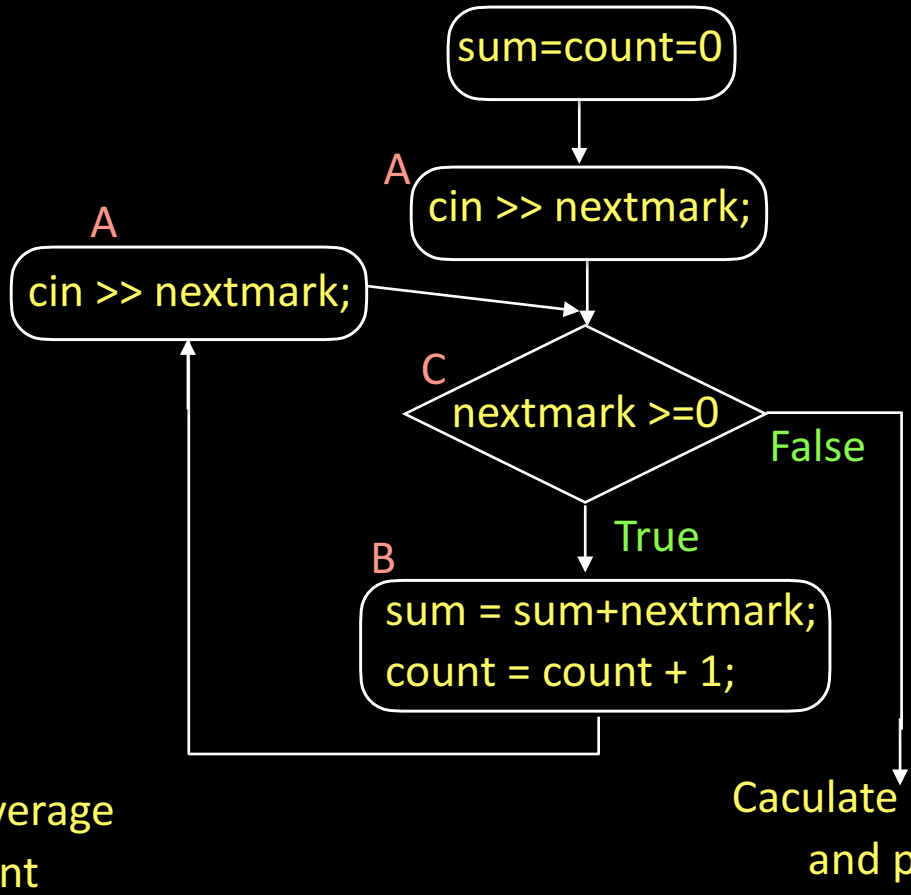
1. Set sum = count = 0.
2. Read next value into nextmark
3. If nextmark < 0, then go to step 7, if it is >= 0, continue to step 4.
4. sum = sum + nextmark
5. count = count + 1
6. Go to step 2.
7. Print sum/count.

Structures does not match with while.

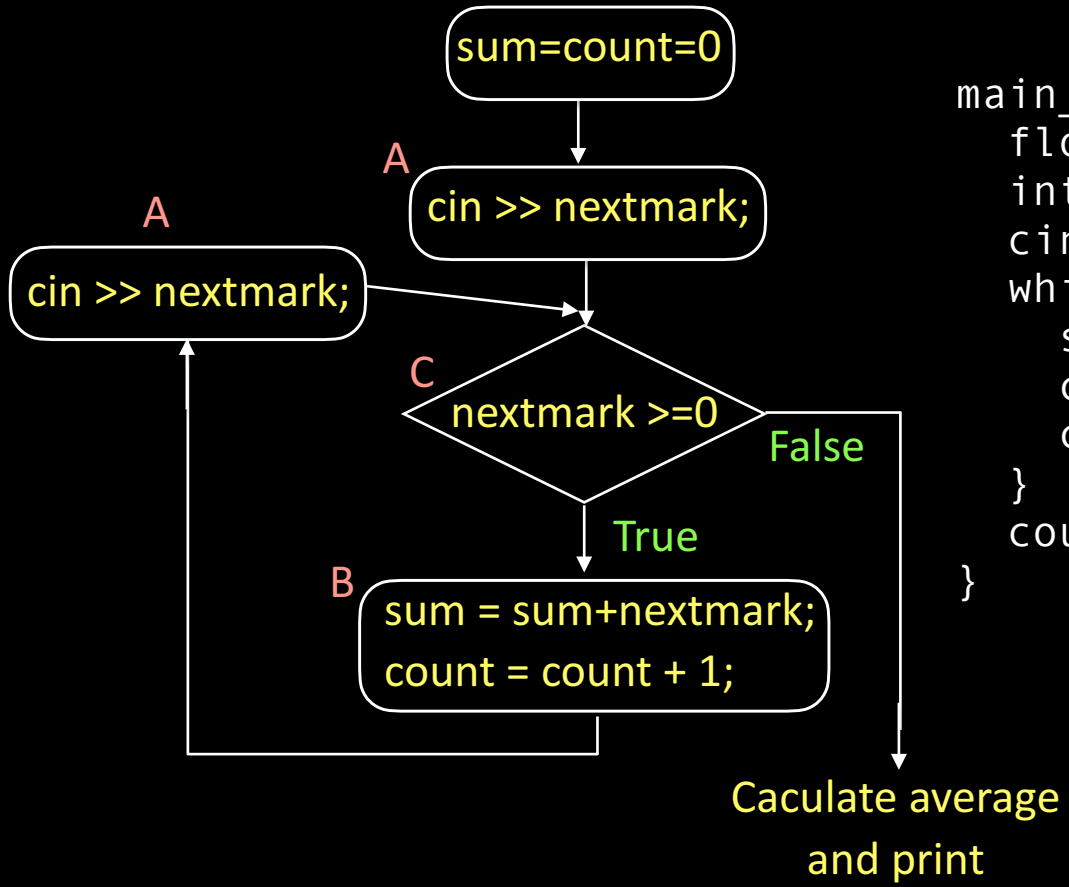


A different flowchart for mark averaging





A different flowchart for mark averaging




```
main_program{
   float nextmark, sum = 0;
   int count = 0;
   cin >> nextmark;          // A
   while(nextmark >= 0){     // C
     sum = sum + nextmark; // B
     count = count + 1;     // B
     cin >> nextmark; // A copy
   }
   cout << sum/count << endl;
}
```

Exercise

Write a turtle controller which receives commands from the keyboard and acts per the following rules:

- If command = ‘f’ : move turtle forward 100 pixels
- If command = ‘r’ : turn turtle right 90 degrees.
- If command = ‘x’ : finish execution.

What we discussed

- In while loops, the first step is to check whether we need to execute the next iteration.
- In some loops that we want to write, the first step is not a condition check. The condition check comes later.
- Such loops can be modified by making a copy of the steps before the condition check.
- NEXT: loops in which condition checks can happen also inside the body

The break statement

Form: The `break` keyword is a statement by itself.

What happens when control reaches break statement:

- The execution of the `while` statement which contains it is terminated.
- The execution continues from the next statement following that `while` statement.

Example of break

```
main_program{
  float nextmark, sum = 0;
  int count = 0;
  while(true){
      cin >> nextmark;
      if(nextmark < 0) break;
      sum += nextmark;
      count++;
  }
  cout << sum/count << endl;
}
```

- The condition of the while statement is given as `true` – body will always be entered.
- If nextmark < 0:
  - the while loop execution will terminate
  - Execution continues from the statement after while, i.e. cout ...
- Exactly what we wanted!
  - No need to copy code.
- Some programmers do not like break statements because continuation condition gets hidden inside body, instead of being at the top.
- Condition for breaking = compliment of condition for continuing loop

The continue statement

- The `continue` is another single word statement.
- If it is encountered in execution:
  - The control directly goes to the beginning of the loop for the next iteration,
  - Statements from the `continue` to the end of the loop body are skipped.

Example

Mark averaging with an additional condition:

- If a number > 100 is read, ignore it.
  - say because marks can only be at most 100
- Continue execution with the next number.
- As before stop and print the average only when a negative number is read.

Code for new mark averaging

```
main_program{
  float nextmark, sum = 0;
  int count = 0;
  while(true){
      cin >> nextmark;
      if(nextmark > 100) continue;
      if(nextmark < 0) break;
      sum += nextmark;
      count++;
  }
  cout << sum/count << endl;
}
```

The do while statement

- Not very common.
- Discussed in the book.

Exercise

- Write the turtle controller program using the break statement.

Requirements:

- ‘f’ : mover forward 100 pixels.
- ‘r’ : right turn 90 degrees.
- ‘x’ : exit program.

What we discussed

- The break statement enables us to exit a loop from inside the body.
- If break appears inside a while statement which is itself nested inside another while, then the inner while statement is terminated.
- The continue statement enables us to skip the rest of the iteration.

The for statement: motivation

- Example: Write a program to print a table of cubes of numbers from 1 to 100.

```
int i = 1;
repeat(100){
   cout << i <<‘ ‘<< i*i*i << endl;
   i++;
}
```

- This idiom: do something for every number between x and y occurs very commonly.
- The `for` statement makes it easy to express this idiom, as follows:

```
for(int i=1; i<= 100; i++)
   cout << i <<‘ ‘<< i*i*i << endl;
```

- We will see how this works next.

The for statement

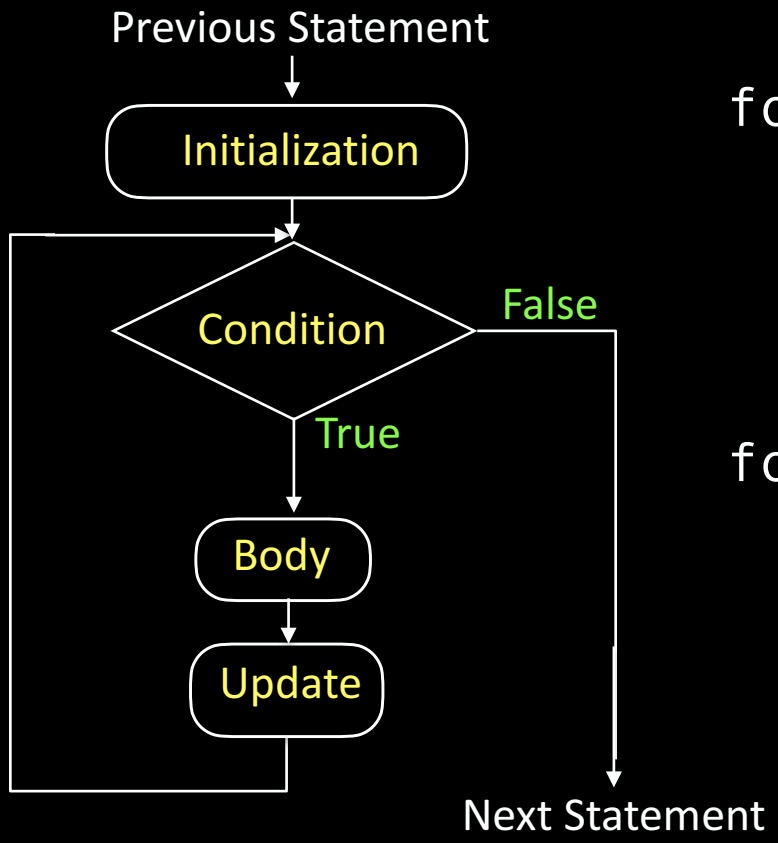
Form: `for(initialization; condition; update) body`

- `initialization, update` : Typically assignments (no semi-colon).
- `condition` : boolean expression.

Execution:

- Before the first iteration of the loop the `initialization` is executed.
- Within each iteration:
  - `condition` is first tested.
  - If it fails, the loop execution ends.
  - If the `condition` succeeds, then the `body` is executed.
  - After that the `update` is executed. Then the next iteration begins.
- Flowchart given next.

Flowchart for for




```
for(initialization;
    condition;
    update)
  body

for(int i=1;
    i <= 100;
    i++)
  cout <<i<<‘ ‘<<i*i*i<<endl;
```

Remarks

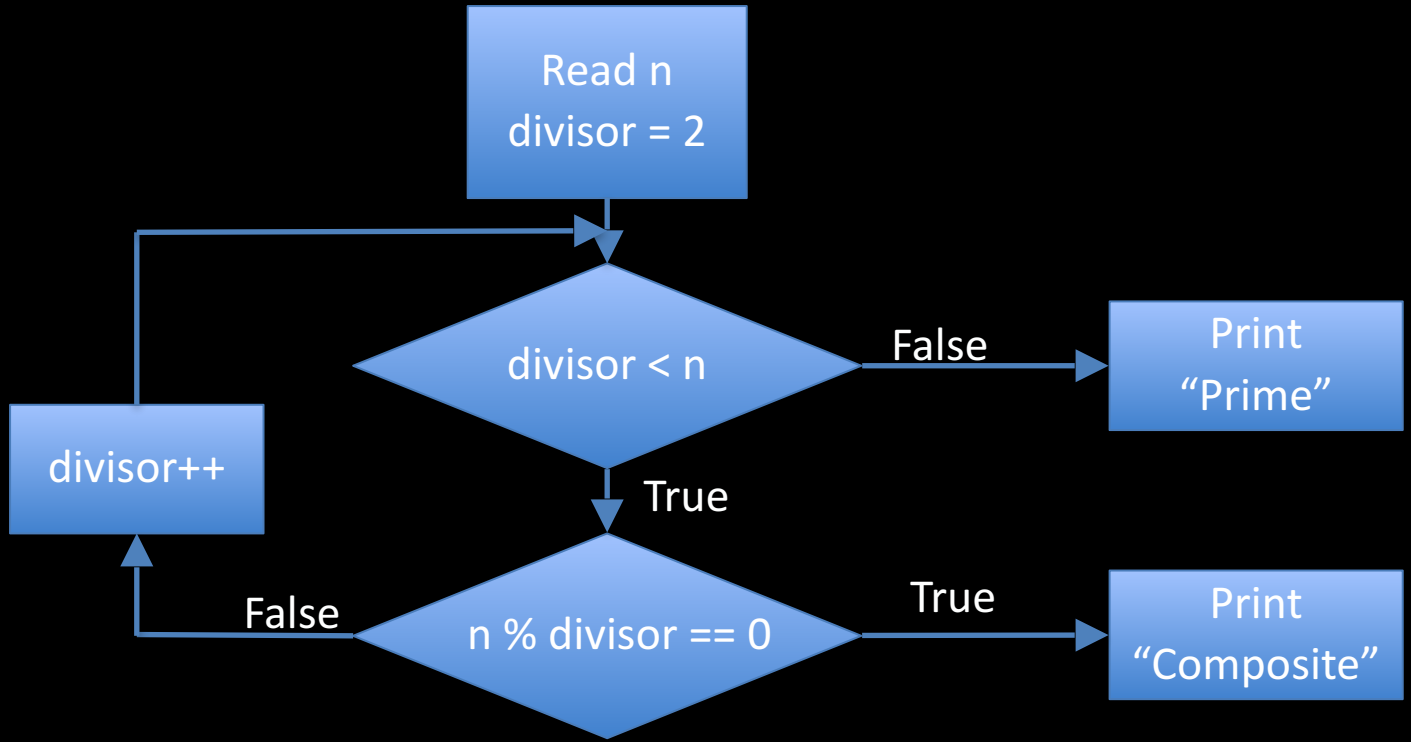
- New variables can be defined in `initialization`. These variables are accessible inside the loop body, including `condition` and `update`, but not outside.
- Variables defined outside can be used inside, unless shadowed by new variables.
- `Break` and `continue` can be used, with natural interpretation.
- Typical use of `for` a single variable is initialized and updated, and the condition tests whether it has reached a certain value. Such a variable is called the control variable of the `for` statement.

Determining whether a number n is prime

Simple manual algorithm: Check whether any of the numbers between 2 and n-1 divides n.

Will improve upon what we did last week.

- Make a flowchart of the manual algorithm.
- See if it can be put into the format of the `for` statement.

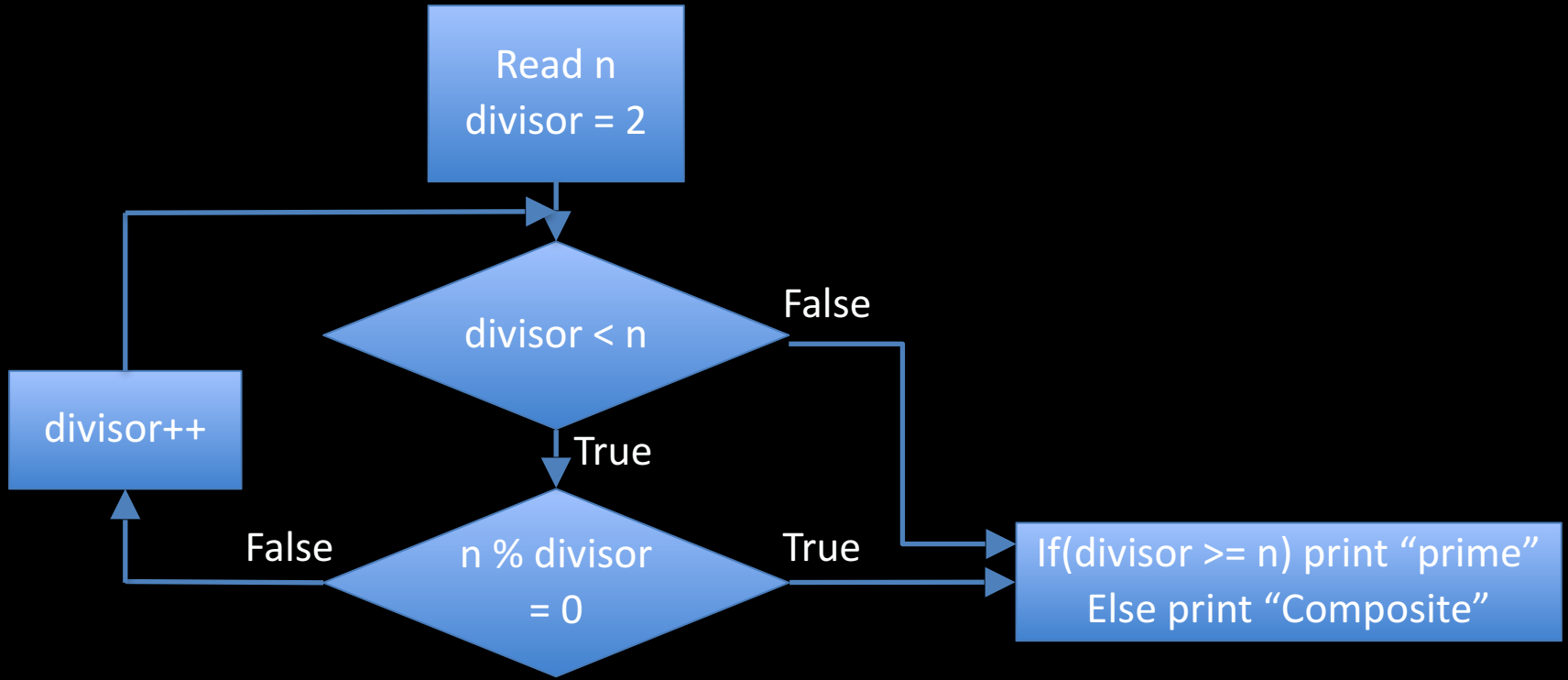

Read n
divisor = 2
divisor < n
False
Print
“Prime”
divisor++
True
False
n % divisor == 0
True
Print
“Composite”

Remarks

- The previous flowchart is functionally correct and faithfully represents what you do manually.

However, flowcharts are expected to be “structured”

- Should not have paths flowing all over the page.
- Typically, the flow should be:
  - Sequence of steps
  - Some of the steps can be loops, which may contain loops..
  - Single start point, single end point
- Our flowchart has two paths going out, i.e. 2 end points.
  - We should try to avoid this.


Read n
divisor = 2
divisor < n
False
True
divisor++
n % divisor
= 0
False
True
If(divisor >= n) print “prime”
Else print “Composite”

Program to test primality

```
main_program{
  int n; cin >> n;
  int divisor = 2;
  for( ; divisor < n; divisor++){
    if(n % divisor == 0) break;
  }
  if(divisor >= n) cout <<“Prime”<<endl;
  else cout <<“Composite”<<endl;
}
```

Remarks

- We have left the “initialization” part of the for statement empty – this is allowed.
- We could have placed divisor = 2 in the initialization.
- However, we could not have placed “int divisor = 2” in the initialization – then the variable divisor would not be available outside the loop, in the last statement.

Exercise: What will this program print?

```
main_program{
  int n; cin >> n;
  int divisor = 2;
  for(int divisor=2 ; divisor < n; divisor++){
    if(n % divisor == 0) break;
  }
  if(divisor >= n) cout <<“Prime”<<endl;
  else cout <<“Composite”<<endl;
}
```

Exercise

- Write a program that prints out the sequence 1, 2, 4, ... 65536.
  - Hint: The update part of the for does not have to be addition, it can be other operations too.

What we discussed

- Often we need to iterate such that in each iteration a certain variable takes a simple sequence of values, i.e. variable i goes form 1 to n.
- In such a case the for statement is very useful
- The variable whose values form the sequence is called a “control variable” for the loop.
- Matching the flow chart of the manual algorithm to the structure of the while or for takes some work.

Saturday extra lab session / Next week

- This saturday 10 am - 12 pm.
- Please come if you are not able to finish lab assignments.
- TAs will be more accessible.

- Next week: work on the lab computers. Familiarize yourself for the lab exam.
- Quiz solutions and marking scheme will be put up on CS101 homepage.

More on variables and data types

Variables and data types

- bool 0 (false) and 1 (true)
  - Every nonzero number is taken as true.
- `const float Avogadro = 6.022e23;`
- `int q = 'a';`
- `char letter_a = 97;`
- `char letter_b = letter_a + 1;`
- `cout<<int(‘a’); //explicit type conversion`
- `(x>y)?1:0`

Variables and data types

- `inf`
  - Obtained by multiplying two large float variables
  - Or dividing by zero
- nan
  - Zero by zero
  - sqrt(-1)
- HUGE_VAL
- `max`

An elaborate programming example

- Uses `while`, cannot be done using `repeat`.
  - Can be done using `for`.
- Algorithm is clever, correctness is not obvious.
- Exercise in algorithm design, including arguing correctness.

Euclid’s algorithm for GCD

- Greatest common divisor (GCD) of positive integers m, n : largest positive integer p that divides both m, n.
- “Standard” method: factorize m,n and multiply common factors.
- Euclid’s algorithm (2300 years old!) is different and much faster.
- Program based on Euclid’s method will be much faster than program based on factoring.

Euclid's Algorithm

Basic Observation: If d divides both m, n, then d divides m-n also, assuming m > n.

- Proof: m=ad, n=bd, so m-n=(a-b)d.

Also true: If d divides m-n and n, then it divides m too.

- Proof: m-n=cd, n=ed, so m = (c+e)d
- m, n, have the same common divisors as m-n,n.
- The largest divisor of m,n is also the largest divisor of m-n,n.

Insight:

- Instead of finding GCD(m,n), we might as well find GCD(n, m-n).

Example

GCD(3977, 943)

=GCD(3977-943,943) = GCD(3034,943)

=GCD(3034-943,943) = GCD(2091,943)

=GCD(2091-943,943) = GCD(1148,943)

=GCD(1148-943,943) = GCD(205, 943)

We should realize at this point that 205 is just 3977 % 943.

So we could have got to this point just in one shot by writing GCD(3977,943) = GCD(3977 % 943, 943)

Example

Should we guess that GCD(m,n) = GCD(m%n, n)?

This is not true if m%n = 0, since we have defined GCD only for positive integers. But we can save the situation, as Euclid did.

Euclid's theorem Let m,n>0 be positive integers. If n divides m then GCD(m,n) = n. Otherwise GCD(m,n) = GCD(m%n, n).

Example continued

GCD(3977,943)

= GCD(3977 % 943, 943)

= GCD(205, 943) = GCD(205, 943%205)

= GCD(205,123) = GCD(205%123,123)

= GCD(82, 123) = GCD(82, 123%82)

= GCD(82, 41)

= 41 because 41 divides 82.

Exercise

- Use Euclid’s algorithm to find the GCD of 26, 42.

Algorithm

- input: values M, N which are stored in variables m, n.
- iteration : Either discover the GCD of M, N, or find smaller numbers whose GCD is same as GCD of M, N.
- Details of an iteration:
  - At the beginning we have numbers stored in m, n, whose GCD is the same as GCD(M,N).
  - If n divides m, then we declare n to be the GCD.
  - If n does not divide m, then we know that GCD(M,N) = GCD(n, m%n).
  - So we have smaller numbers n, m%n, whose GCD is same as GCD(M,N)

Program for GCD

```
main_program{
   int m, n; cin >> m >> n;
   while(m % n != 0){
      int nextm = n;
      int nextn = m % n;
      m = nextm;
      n = nextn;
   }
   cout << n << endl;
}
// To store n, m%n into m,n, we cannot
// just write m=n; n=m%n;
// Can you say why?  Hint: take an example!
```

Remark

- We have defined variables `nextm, nextn` for clarity. We could have done the assignment with just one variable as follows.

```
int r = m%n; m = n; n = r;
```

- It should be intuitively clear that in writing the program, we have followed the idea from Euclid's theorem.
- However, it is possible to make a mistake in "following the idea"...

Exercise: Find the mistake in this program

```
main_program{
   int m, n; cin >> m >> n;
   while(m % n != 0){
      int nextm = n;
      int nextn = m % n;
      m = nextn;
      n = nextm;
   }
   cout << n << endl;
}
```

What we discussed

- Euclid's algorithm for determining the GCD of two positive integers.
- Program appears reasonably easy to write, but there is room to make mistakes.
- NEXT: How do we check correctness of our program and avoid mistakes.

Correctness proof

- We should have a proof that we have implemented major ideas correctly, and also not made any silly mistakes.
- If the program is straight line code, i.e. no loops, correctness proof is usually simple: claim what happens after each statement is executed.
- But if the statements are in a loop, such claims are harder to make and prove.
  - Need some special machinery

Correctness proof for a program

- Need to prove that the program
  - Terminates: does not go into an endless loop
  - Gives the correct answer on termination
- Typically done by defining
  - “Potential”
  - “Invariant”

Invariants

- Let M, N be the values typed in by the user into variables `m` , `n`.
- We can make the following claim.

Just before and just after every iteration, the values of m, n might be different, however, GCD(`m` , n) = GCD(M,N). Further m,n>0.

- Such a statement which says that certain property does not change due to the loop iteration is called a loop invariant, or invariant for short.

Proof sketch of invariant: GCD(m,n)=GCD(M,N) at the beginning and end of every iteration

```
main_program{
  int m, n;
  cin >> m >> n;
  while(m % n != 0){
      int nextm = n;
      int nextn = m % n;
      m = nextm;
      n = nextn;
  }
  cout << n << endl;
}
```

Proof of termination

```
main_program{
   int m, n; cin >> m >> n;
   while(m % n != 0){
      int nextm = n;
      int nextn = m % n;
      m = nextm;
      n = nextn;
   }
   cout << n << endl;
}
```

Termination + Invariant = correctness

- The algorithm terminates, so `m%n` must have been 0.
- But in this case GCD(`m` , `n`) = `n`, which is what the algorithm prints.
- But this is correct because by the invariant, GCD(`m` , `n`) = GCD(M,N) which is what we wanted.

Summary of proof strategy

- In each iteration, m,n may change, but their GCD does not change.
  - This helped us argue that the eventual value will be correct.
- In each iteration n reduces but can never become 0.
  - Since it has to reduce by at least one, number of iterations < n.

Another example: cube table program

```
for(int i=1; i<=100; i++)
   cout << i <<‘ ‘<<i*i*i<<endl;
```

- Invariant: when iteration i begind cubes until `i` - 1 have been printed.
  - True for every iteration!
- Potential: value of `i` : it must increase in every step, but cannot increase beyond 100.
- For programs so simple, writing invariants seems to make simple things unnecessarily complex.
  - Invariant does say something useful: how the program makes progress.

Remarks

- `while, do while, for` are the C++ statements that allow you to loop.
- `repeat` allows you to loop, but it is not a part of C++. It is a part of simplecpp; it was introduced because it is very easy to understand. Now that you know `while, do while, for`, you can stop using `repeat`.

Concluding Remarks

The important issues in writing a loop are

- Ensure that you do not execute iterations indefinitely.
- If you cannot terminate the loop using a check at the beginning of the repeated portion, you can either duplicate code, or use `break`.
- The remarks in Chapter 3 about thinking of the manual algorithm, making a plan and identifying the general pattern of actions apply here also.
- The actions in the loop may include preparing for the next iteration, e.g. incrementing a variable, or setting new values of `m`, `n` as in the GCD program.
- Think about the invariants and the potential. This is a good way to cross-check that your loop is doing the right thing, and that it will terminate. Write these down explicitly when the algorithm is even slightly clever.
- Solve the problems at the end of Chapter 7.

Computing π

- Compute the area of a circle of radius 1 as follows:
  - Divide the circle into, say 10000 vertical strips.
  - Assume each vertical strip is a rectangle.
  - The area of each strip can be computed by height*width.
  - The total area of all strips gives us the area of the circle.

Computing π

```
#include<simplecpp>
#include<iomanip>

main_program{
  double noOfStrips;
  cin >> noOfStrips;

  double width = 1/noOfStrips;
  double area = 0;

  for(int i=1; i<=noOfStrips; i++){
    double x = (i)*width;
    double height = sqrt(1-x*x);
    area = area + height*width;
  }
  cout << setprecision(15);
  cout << area*4 << endl;
}
```

iomanip is for setting precision for printing double variables

noOfStrips is the number of vertical strips into which the half circle is divided

x is x-distance of the i-th strip from center. height is the height above center line.

the loop computes the area of a quarter circle.